Qué subir a GitHub

Ingeniería de Métodos | Preparación del piloto | 9 de noviembre de 2026

1. La carpeta correcta

Usa **ingenieria-metodos-app** como raíz del repositorio. Dentro deben verse directamente **backend/** y **frontend/**, junto con README.md, .gitignore, .python-version y render.yaml. No subas la carpeta completa de Servicio Social ni los materiales de DesignTools, respaldos o documentos ajenos a la aplicación.

La aplicación está hecha con **FastAPI**, HTML, CSS y JavaScript. El backend ejecuta los cálculos y también sirve las páginas. No es Flask. En esta versión, los estudios se conservan en el navegador y en archivos descargables; no hay conexión a PostgreSQL, cuentas de usuario ni uso de SECRET_KEY.

2. Lista exacta de archivos

La siguiente lista describe la entrega preparada. **manifesto-github.txt** contiene todas las rutas individuales y coincide con el contenido del ZIP limpio. Incluye documentación y pruebas, aunque estas no sean necesarias para arrancar el servidor.

En la raíz:

```
.gitignore
.env.example
.python-version
render.yaml
README.md
manifesto-github.txt
```

En backend/:

```
main.py
calculos.py
schemas.py
suplementos.py
westinghouse.py
estudios.py
estudios_schemas.py
hombre_maquina.py
requirements.txt
```

En frontend/:

```
index.html
style.css
exploratorias.html
exploratorias-hub.css
ishikawa.html
ishikawa.css
ishikawa.js
ishikawa-export.js
pareto.html
pareto.css
pareto.js
diagrama-flujo.html
diagrama-flujo.css
diagrama-flujo.js
pdf-diagrama.js
```

```
simbolos.js
recorrido.html
recorrido.css
recorrido.js
operador-maquina.html
operador-maquina.js
registro-analisis.html
registro-analisis.css
registro-analisis.js
balanceo-lineas.html
balanceo-lineas.js
muestreo-trabajo.html
muestreo-trabajo.js
estudios-comun.js
estudios.css
persistencia.js
persistencia.css
persistencia-diagramas.js
```

El archivo operador-maquina conserva su nombre interno para mantener enlaces y compatibilidad. La interfaz muestra Hombre-Máquina; no lo renombres por separado.

Documentación y pruebas:

```
docs/01-guia-github.md
docs/02-guia-render.md
docs/03-manual-usuario.md
docs/hombre-maquina.md
docs/metodos-implementados.md
docs/capturas/inicio-simplificado.png
output/pdf/01-guia-github.pdf
output/pdf/02-guia-render.pdf
output/pdf/03-manual-usuario.pdf
tests/api_inprocess.py
tests/frontend-estudios.cjs
tests/persistencia.cjs
tests/test_estudios.py
tests/test_hombre_maquina.py
```

3. Qué NO subir

No subir	Motivo
.env y .env.* con valores reales	Pueden contener contraseñas, tokens y conexiones privadas.
venv/, backend/venv/, .venv/	Son instalaciones locales; Render crea su propio entorno.
__pycache__/, *.pyc y cachés de pruebas	Archivos generados por Python y las herramientas.
*.pem, *.key, credentials.json, secrets/	Claves privadas y credenciales.
Bases locales, *.dump y respaldos	Pueden contener información del estudio o de participantes.
datos-piloto/, estudios-guardados/, uploads/	Datos reales; no pertenecen al código fuente.
work/, tmp/, ZIP de respaldo y DesignTools	Material de trabajo, entregas comprimidas o software externo.
.DS_Store, registros y ajustes del editor	Archivos del equipo, sin utilidad para ejecutar la aplicación.

El .gitignore ya contiene estas exclusiones. **.env.example sí se sube**: solo contiene instrucciones comentadas, sin valores secretos. Guarda los estudios reales fuera del repositorio o en las carpetas excluidas; un archivo JSON con un nombre arbitrario no se detecta automáticamente como privado.

El .gitignore no retira archivos que Git ya estuviera siguiendo. Si alguna credencial llegó a un commit, hay que revocarla o cambiarla y corregir el historial según corresponda. Esta carpeta todavía no tenía repositorio Git al preparar la entrega. Referencia: documentación de gitignore.

4. Archivos necesarios para Render

Archivo o herramienta	Función en este proyecto
backend/requirements.txt	Dependencias: FastAPI, Uvicorn y Pydantic. Se fijaron las versiones directas verificadas.
backend/main.py y módulos importados	Contienen la aplicación y los cálculos.
frontend/ completo	Páginas, estilos, diagramas, exportación y guardado.
.python-version	Selecciona Python 3.14.7, la versión usada en las pruebas locales.
render.yaml	Configuración reproducible para un Blueprint; opcional si configuras el servicio manualmente.
Procfile	No hace falta con el Start Command configurado en Render.
Gunicorn	No hace falta para esta configuración: se ejecuta Uvicorn.

Las dependencias indirectas siguen siendo resueltas por pip; fijar las tres dependencias directas no equivale a un bloqueo completo de todas las versiones. La instalación final se valida también en los registros de Render.

5. Crear el repositorio y subir la copia limpia

1. Descomprime **ingenieria-metodos-github.zip**. La carpeta resultante ya excluye los entornos, cachés y archivos privados habituales.
2. Crea un repositorio llamado, por ejemplo, **ingenieria-metodos-app**. Puedes mantenerlo privado y autorizar a Render a leer únicamente ese repositorio.
3. Usa la carpeta descomprimida como raíz del repositorio. Si creas el repositorio vacío, no añadas otro README o .gitignore desde GitHub.
4. Publica los archivos con tu herramienta habitual de Git. No subas el ZIP como un único archivo: Render necesita ver backend/ y frontend/ descomprimidos.
5. Antes del primer commit, revisa la lista de cambios. En macOS, los archivos que empiezan con punto pueden estar ocultos; comprueba que .gitignore y .python-version sí se incluyeron.
6. En GitHub, verifica que puedas abrir backend/main.py, backend/requirements.txt, frontend/index.html y render.yaml.

Si utilizas terminal, desde la carpeta del repositorio puedes revisar las exclusiones con:

```
git status --short
git check-ignore -v .env backend/venv/ejemplo __pycache__/ejemplo.pyc
```

El segundo comando debe mostrar reglas de exclusión para esos ejemplos. Revisa también que no aparezcan datos reales entre los archivos nuevos. Continúa con **02-guia-render.pdf**.